

The Anatomy of a Website

Browsers, your first HTML, and the boilerplate — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Keep your **browser** and your **Terminal** open beside you. For every task, **attempt and write your answer before we discuss it**. Today we open up a real website, peek at the HTML behind it, and write our very first page.

Part 1 — The Browser

Q1. Which **web browser** do you use most often? Why do you prefer it over the others?

Q2. List at least **five web browsers** you know. Which **company** develops each one?

Browser	Developed by
_____	_____
_____	_____
_____	_____
_____	_____
_____	_____

Q3. What **is** a browser? What sort of things do you generally **do** using a browser?

Q4. Let's talk about **Chrome**. Have you heard of it? I'll be using it throughout the course. List **5 pros** of using Chrome.

TRY THIS · CHROME

Make **Chrome your default browser**. (Settings → "Default browser" → *Make default*.) From now on, links you click will open in Chrome.

Q5. You already know about **domains** from earlier sessions. Open **three popular websites** using their domains in **three different tabs**. What do you see? (e.g. `youtube.com`, `flipkart.com`, `zenithschool.ai`)

TRY THIS · BROWSER

Right-click anywhere on the page — this opens a *context menu* — and click "**View Page Source**". Repeat this for **all three** websites.

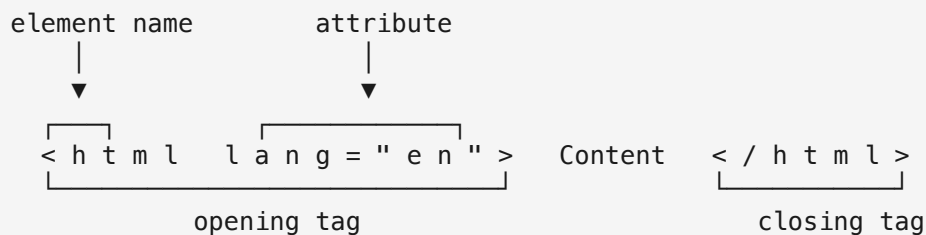
Q6. What do you see here? Look at the **first few lines** of each — what looks **common** across all three?

Part 2 — First Interaction with HTML

The output you just saw is **HTML**. The very first line, `<!DOCTYPE html>`, is a **document type declaration**. It tells the browser you are using **HTML5**, the latest version of HTML.

Q7. If I tell you that an **HTML tag** is enclosed within `<` and `>`, which HTML tags are you able to **identify** in the page source right now?

We have different **HTML elements**. Each element usually has **4 main parts**: an **opening tag**, **attribute(s)**, **content**, and a **closing tag**.



Opening Tag: marks the *start* of the element — the element name in angle brackets (e.g. `<p>`).

Attributes: optional properties placed inside the opening tag to give extra info or configuration.

Content: the actual text, image, or media shown on the screen.

Closing Tag: marks the *end* of the element — identical to the opening tag but with a forward slash (e.g. `</p>`).

Q8. Now — what do you think is the difference between an **Element** and a **Tag**?

Q9. Do you see any elements that have **no closing tag**? If yes, name them.

TRY THIS · TERMINAL

Let's create our own HTML page.

```
mkdir Full_Stack_Sessions # create a new folder
cd Full_Stack_Sessions
touch 1.html # create an empty file
vim 1.html # open it in vim
```

Inside `1.html`, type:

```
<h1>Hello!</h1><p>I am Likhilesh!</p>
```

Save and exit vim (`:wq`), then run:

```
open 1.html
```

Q10. What do you see? Open the **Page Source** and **relate it** to the output on screen.

TRY THIS · BROWSER

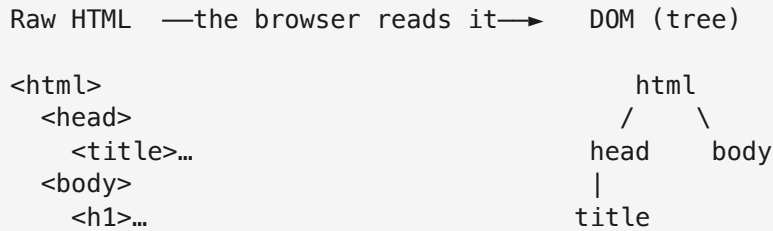
Open the context menu again (**right-click** anywhere on the page), but this time click **"Inspect"**. This opens the **Developer Tools** area, with tabs like `Elements`, `Console`, `Network`, `Application`, ... This area is built specially for developers — hence the name.

Q11. What all did you see in the Developer Tools? Is there a specific area where you see something that looks like **HTML**?

Q12. Pay close attention: what is **different** between what you see in **"View Page Source"** and what you see in the **"Elements"** tab inside Developer Tools?

Part 3 — Understanding the HTML Boilerplate

The browser receives the initial **HTML** and reads it. Side by side, it builds a **tree-like structure** called the **DOM**. The *live view* of this DOM is exactly what the **Elements** tab shows. We'll talk a lot more about the DOM when we get to JavaScript.



Q13. How will you write your code when you build **big websites / web-applications**? How will you **manage** it? You've already heard of **Git** — what else?

Q14. What **IDEs** have you heard of? What **problems** do they solve?

Q15. We'll use **Visual Studio Code**, the gold standard of web development. Have you heard of it? Write **2 reasons** why I might call it the "Gold Standard".

TRY THIS · VS CODE

Open **Visual Studio Code**. Click "**Open Folder**" and open the `Full_Stack_Sessions` folder you created earlier.

Q16. Do you see the `1.html` file present in the folder now?

TRY THIS · VS CODE

Click on `1.html` on the left. **Remove all of its content** — make it blank. Now type `!` (an exclamation mark); a suggestion appears. Press **Enter** after you see the suggestion.

Q17. What do you see now? This is called the **Boilerplate code** of HTML. Write your observations here.

Q18. You can see `html` is the **parent** element, while `head` & `body` are its **children**. Draw an **upside-down tree** diagram of this — start from `html` and include **everything** you see in the boilerplate.



TRY THIS · VS CODE + BROWSER

In between the **opening and closing tag of the body element**, write the **same code** we wrote earlier in `1.html`, then **save** the file:

```
<h1>Hello!</h1><p>I am Likhilesh!</p>
```

In the left sidebar, **right-click** on the filename `1.html` and select **"Reveal in Finder"** (or "Reveal in File Explorer"). Your file manager opens — **double-click** the file there. It opens as a webpage in your **default browser**. *(I recommend making Chrome your default browser if you haven't already.)*

Q19. What is the difference between the **"Page Source"** you can see now compared to earlier? Is the **output on the screen** (the web page) different?

Q20. Open the **"Elements"** tab in Developer Tools — does it look the **same** as earlier, or is there any **change**?

CLASS WRAP-UP · TERMINAL + GIT

In VS Code's left sidebar, use the **"New Folder"** button to create a folder named `1_HTML`, then **cut and paste** `1.html` into it — this is just an arrangement so you can organise your files conveniently in future sessions. Now link the root folder `Full_Stack_Sessions` to a new **"Public"** git repository (keep the GitHub repository name `Full_Stack_Sessions`), then **add, commit** and **push** your code to GitHub:

```
git init
git add .
git commit -m "First HTML page"
git push
```

TRY THIS · GITHUB PAGES

Let's put your page on the internet. Open your GitHub repository and:

1. Go to the **Settings** tab (last item in the same row as the **Code** tab).
2. Click **Pages** in the left sidebar.
3. Under "**Deploy from a branch**", select **GitHub Actions**.
4. In the first card, "**GitHub Pages Jekyll**", click **Configure**.
5. Click the green "**Commit changes...**" button, then click "**Commit changes**" once more to confirm.
6. Go to the **Actions** tab (also at the end of the **Code** row). You'll see a deployment "**Queued / In progress**" (yellow). It completes in roughly **50 seconds** and the marker turns **green**.
7. Go back to **Settings** → **Pages** — a **link** now appears that wasn't there before. Open it.

Q21. Do you see any output on the webpage, or is it an **error page**?

TRY THIS · BROWSER

After your link, type `1_HTML/1.html` — does this show any output? The link acts as a **base** that redirects to your repository. Your HTML file lives inside the `1_HTML` folder and is named `1.html`, so the full path is *link + folder + file name*:

```
<link>/1_HTML/1.html
```

Q22. Share this link with a friend on **WhatsApp** and ask them to open it and see the output.

Note: You made those changes on GitHub, which created a `.github` folder on the **remote** — but that folder is not yet on your local machine. So you need to **pull** those changes down:

```
git pull
```